

72

2.5.3 节练习

练习 2.36: 关于下面的代码, 请指出每一个变量的类型以及程序结束时它们各自的值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype((b)) d = a;
++c;
++d;
```

练习 2.37: 赋值是会产生引用的一类典型表达式, 引用的类型就是左值的类型。也就是说, 如果 `i` 是 `int`, 则表达式 `i=x` 的类型是 `int&`。根据这一特点, 请指出下面的代码中每一个变量的类型和值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype(a = b) d = a;
```

练习 2.38: 说明由 `decltype` 指定类型和由 `auto` 指定类型有何区别。请举出一个例子, `decltype` 指定的类型与 `auto` 指定的类型一样; 再举一个例子, `decltype` 指定的类型与 `auto` 指定的类型不一样。



2.6 自定义数据结构

从最基本的层面理解, 数据结构是把一组相关的数据元素组织起来然后使用它们的策略和方法。举一个例子, 我们的 `Sales_item` 类把书本的 ISBN 编号、售出量及销售收入等数据组织在了一起, 并且提供诸如 `isbn` 函数、`>>`、`<<`、`+`、`+=` 等运算在内的一系列操作, `Sales_item` 类就是一个数据结构。

C++ 语言允许用户以类的形式自定义数据类型, 而库类型 `string`、`istream`、`ostream` 等也都是以类的形式定义的, 就像第 1 章的 `Sales_item` 类型一样。C++ 语言对类的支持甚多, 事实上本书的 III 部分和 IV 部分都将大篇幅地介绍与类有关的知识。尽管 `Sales_item` 类非常简单, 但是要想给出它的完整定义可在第 14 章介绍自定义运算符之后。



2.6.1 定义 `Sales_data` 类型

尽管我们还写不出完整的 `Sales_item` 类, 但是可以尝试着把那些数据元素组织到一起形成一个简单点儿的类。初步的想法是用户能直接访问其中的数据元素, 也能实现一些基本的操作。

既然我们筹划的这个数据结构不带有任何运算功能, 不妨把它命名为 `Sales_data` 以示与 `Sales_item` 的区别。 `Sales_data` 初步定义如下:

73

```
struct Sales_data {
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

我们的类以关键字 **struct** 开始, 紧跟着类名和类体 (其中类体部分可以为空)。类体由

花括号包围形成了一个新的作用域（参见 2.2.4 节，第 43 页）。类内部定义的名字必须唯一，但是可以与类外部定义的名字重复。

类体右侧的表示结束的花括号后必须写一个分号，这是因为类体后面可以紧跟变量名以示对该类型对象的定义，所以分号必不可少：

```
struct Sales_data { /* ... */ } accum, trans, *salesptr;
// 与上一条语句等价，但可能更好一些
struct Sales_data { /* ... */ };
Sales_data accum, trans, *salesptr;
```

分号表示声明符（通常为空）的结束。一般来说，最好不要把对象的定义和类的定义放在一起。这么做无异于把两种不同实体的定义混在了一条语句里，一会儿定义类，一会儿又定义变量，显然这是一种不被建议的行为。



很多新手程序员经常忘了在类定义的最后加上分号。

类数据成员

类体定义类的成员，我们的类只有数据成员（data member）。类的数据成员定义了类的对象的具体内容，每个对象有自己的一份数据成员拷贝。修改一个对象的数据成员，不会影响其他 Sales_data 的对象。

定义数据成员的方法和定义普通变量一样：首先说明一个基本类型，随后紧跟一个或多个声明符。我们的类有 3 个数据成员：一个名为 bookNo 的 string 成员、一个名为 units_sold 的 unsigned 成员和一个名为 revenue 的 double 成员。每个 Sales_data 的对象都将包括这 3 个数据成员。

C++11 新标准规定，可以为数据成员提供一个类内初始值（in-class initializer）。创建对象时，类内初始值将用于初始化数据成员。没有初始值的成员将被默认初始化（参见 2.2.1 节，第 40 页）。因此当定义 Sales_data 的对象时，units_sold 和 revenue 都将初始化为 0，bookNo 将初始化为空字符串。

C++
11

对类内初始值的限制与之前（参见 2.2.1 节，第 39 页）介绍的类似：或者放在花括号里，或者放在等号右边，记住不能使用圆括号。

7.2 节（第 240 页）将要介绍，用户可以使用 C++ 语言提供的另外一个关键字 class 来定义自己的数据结构，到时也将说明现在我们使用 struct 的原因。在第 7 章学习与 class 有关的知识之前，建议读者继续使用 struct 定义自己的数据类型。

2.6.1 节练习

74

练习 2.39: 编译下面的程序观察其运行结果，注意，如果忘记写类定义体后面的分号会发生什么情况？记录下相关信息，以后可能会有用。

```
struct Foo { /* 此处为空 */ } // 注意：没有分号
int main()
{
    return 0;
}
```

练习 2.40: 根据自己的理解写出 Sales_data 类，最好与书中的例子有所区别。



2.6.2 使用 Sales_data 类

和 Sales_item 类不同的是，我们自定义的 Sales_data 类没有提供任何操作，Sales_data 类的使用者如果想执行什么操作就必须自己动手实现。例如，我们将参照 1.5.2 节（第 20 页）的例子写一段程序实现求两次交易相加结果的功能。程序的输入是下面这两条交易记录：

```
0-201-78345-X 3 20.00
0-201-78345-X 2 25.00
```

每笔交易记录着图书的 ISBN 编号、售出数量和售出单价。

添加两个 Sales_data 对象

因为 Sales_data 类没有提供任何操作，所以我们必须自己编码实现输入、输出和相加的功能。假设已知 Sales_data 类定义于 Sales_data.h 文件内，2.6.3 节（第 67 页）将详细介绍定义头文件的方法。

因为程序比较长，所以接下来分成几部分介绍。总的来说，程序的结构如下：

```
#include <iostream>
#include <string>
#include "Sales_data.h"
int main()
{
    Sales_data data1, data2;
    // 读入 data1 和 data2 的代码
    // 检查 data1 和 data2 的 ISBN 是否相同的代码
    // 如果相同，求 data1 和 data2 的总和
}
```

和原来的程序一样，先把所需的头文件包含进来并且定义变量用于接受输入。和 Sales_item 类不同的是，新程序还包含了 string 头文件，因为我们的代码中将用到 string 类型的成员变量 bookNo。

75 Sales_data 对象读入数据

第 3 章和第 10 章将详细介绍 string 类型的细节，在此之前，我们先了解一点儿关于 string 的知识以便定义和使用我们的 ISBN 成员。string 类型其实就是字符的序列，它的操作有 >>、<< 和 == 等，功能分别是读入字符串、写出字符串和比较字符串。这样我们就能书写代码读入第一笔交易了：

```
double price = 0; // 书的单价，用于计算销售收入
// 读入第 1 笔交易：ISBN、销售数量、单价
std::cin >> data1.bookNo >> data1.units_sold >> price;
// 计算销售收入
data1.revenue = data1.units_sold * price;
```

交易信息记录的是书售出的单价，而数据结构存储的是一次交易的销售收入，因此需要将单价读入到 double 变量 price，然后再计算销售收入 revenue。输入语句

```
std::cin >> data1.bookNo >> data1.units_sold >> price;
```

使用点操作符（参见 1.5.2 节，第 20 页）读入对象 data1 的 bookNo 成员和 units_sold 成员。

最后一条语句把 data1.units_sold 和 price 的乘积赋值给 data1 的 revenue 成员。

接下来程序重复上述过程读入对象 data2 的数据：

```
// 读入第 2 笔交易
std::cin >> data2.bookNo >> data2.units_sold >> price;
data2.revenue = data2.units_sold * price;
```

输出两个 Sales_data 对象的和

剩下的工作就是检查两笔交易涉及的 ISBN 编号是否相同了。如果相同输出它们的和，否则输出一条报错信息：

```
if (data1.bookNo == data2.bookNo) {
    unsigned totalCnt = data1.units_sold + data2.units_sold;
    double totalRevenue = data1.revenue + data2.revenue;
    // 输出：ISBN、总销售量、总销售额、平均价格
    std::cout << data1.bookNo << " " << totalCnt
               << " " << totalRevenue << " ";
    if (totalCnt != 0)
        std::cout << totalRevenue/totalCnt << std::endl;
    else
        std::cout << "(no sales)" << std::endl;
    return 0;          // 标示成功
} else {              // 两笔交易的 ISBN 不一样
    std::cerr << "Data must refer to the same ISBN"
               << std::endl;
    return -1;        // 标示失败
}
```

在第一个 if 语句中比较了 data1 和 data2 的 bookNo 成员是否相同。如果相同则执行第一个 if 语句花括号内的操作，首先计算 units_sold 的和并赋给变量 totalCnt，然后计算 revenue 的和并赋给变量 totalRevenue，输出这些值。接下来检查是否确实售出了书籍，如果是，计算并输出每本书的平均价格；如果售量为零，输出一条相应的信息。

◀ 76

2.6.2 节练习

练习 2.41：使用你自己的 Sales_data 类重写 1.5.1 节（第 20 页）、1.5.2 节（第 21 页）和 1.6 节（第 22 页）的练习。眼下先把 Sales_data 类的定义和 main 函数放在同一个文件里。

2.6.3 编写自己的头文件



尽管如 19.7 节（第 754 页）所讲可以在函数体内定义类，但是这样的类毕竟受到了一些限制。所以，类一般都不定义在函数体内。当在函数体外部定义类时，在各个指定的源文件中可能只有一处该类的定义。而且，如果要在不同文件中使用同一个类，类的定义就必须保持一致。

为了确保各个文件中类的定义一致，类通常被定义在头文件中，而且类所在头文件的名字应与类的名字一样。例如，库类型 string 在名为 string 的头文件中定义。又如，我们应该把 Sales_data 类定义在名为 Sales_data.h 的头文件中。

头文件通常包含那些只能被定义一次的实体，如类、const 和 constexpr 变量（参见 2.4 节，第 54 页）等。头文件也经常用到其他头文件的功能。例如，我们的 Sales_data 类包含有一个 string 成员，所以 Sales_data.h 必须包含 string.h 头文件。同时，使用 Sales_data 类的程序为了能操作 bookNo 成员需要再一次包含 string.h 头文件。

这样，事实上使用 `Sales_data` 类的程序就先后两次包含了 `string.h` 头文件：一次是直接包含的，另有一次是随着包含 `Sales_data.h` 被隐式地包含进来的。有必要在书写头文件时做适当处理，使其遇到多次包含的情况也能安全和正常地工作。



头文件一旦改变，相关的源文件必须重新编译以获取更新过的声明。

预处理器概述

77 确保头文件多次包含仍能安全工作的常用技术是**预处理器**（preprocessor），它由 C++ 语言从 C 语言继承而来。预处理器是在编译之前执行的一段程序，可以部分地改变我们所写的程序。之前已经用到了一项预处理功能 `#include`，当预处理器看到 `#include` 标记时就会用指定的头文件的内容代替 `#include`。

C++ 程序还会用到的一项预处理功能是**头文件保护符**（header guard），头文件保护符依赖于预处理变量（参见 2.3.2 节，第 48 页）。预处理变量有两种状态：已定义和未定义。**`#define`** 指令把一个名字设定为预处理变量，另外两个指令则分别检查某个指定的预处理变量是否已经定义：**`#ifndef`** 当且仅当变量已定义时为真，**`#ifn`**def 当且仅当变量未定义时为真。一旦检查结果为真，则执行后续操作直至遇到 **`#endif`** 指令为止。

使用这些功能就能有效地防止重复包含的发生：

```
#ifndef SALES_DATA_H
#define SALES_DATA_H
#include <string>
struct Sales_data {
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
#endif
```

第一次包含 `Sales_data.h` 时，`#ifndef` 的检查结果为真，预处理器将顺序执行后面的操作直至遇到 `#endif` 为止。此时，预处理变量 `SALES_DATA_H` 的值将变为已定义，而且 `Sales_data.h` 也会被拷贝到我们的程序中。后面如果再一次包含 `Sales_data.h`，则 `#ifndef` 的检查结果将为假，编译器将忽略 `#ifndef` 到 `#endif` 之间的部分。



预处理变量无视 C++ 语言中关于作用域的规则。

整个程序中的预处理变量包括头文件保护符必须唯一，通常的做法是基于头文件中类的名字来构建保护符的名字，以确保其唯一性。为了避免与程序中的其他实体发生名字冲突，一般把预处理变量的名字全部大写。



头文件即使（目前还）没有被包含在任何其他头文件中，也应该设置保护符。头文件保护符很简单，程序员只要习惯性地加上就可以了，没必要太在乎你的程序到底需不需要。

2.6.3 节练习

练习 2.42：根据你自己的理解重写一个 `Sales_data.h` 头文件，并以此为基础重做 2.6.2 节（第 67 页）的练习。

小结

78

类型是 C++ 编程的基础。

类型规定了其对象的存储要求和所能执行的操作。C++ 语言提供了一套基础内置类型，如 `int` 和 `char` 等，这些类型与实现它们的机器硬件密切相关。类型分为非常量和常量，一个常量对象必须初始化，而且一旦初始化其值就不能再改变。此外，还可以定义复合类型，如指针和引用等。复合类型的定义以其他类型为基础。

C++ 语言允许用户以类的形式自定义类型。C++ 库通过类提供了一套高级抽象类型，如输入输出和 `string` 等。

术语表

地址 (address) 是一个数字，根据它可以找到内存中的一个字节。

别名声明 (alias declaration) 为另外一种类型定义一个同义词：使用“名字=类型”的格式将名字作为该类型的同义词。

算术类型 (arithmetic type) 布尔值、字符、整数、浮点数等内置类型。

数组 (array) 是一种数据结构，存放着一组未命名的对象，可以通过索引来访问这些对象。3.5 节将详细介绍数组的知识。

auto 是一个类型说明符，通过变量的初始值来推断变量的类型。

基本类型 (base type) 是类型说明符，可用 `const` 修饰，在声明语句中位于声明符之前。基本类型提供了最常见的数据类型，以此为基础构建声明符。

绑定 (bind) 令某个名字与给定的实体关联在一起，使用该名字也就是使用该实体。例如，引用就是将某个名字与某个对象绑定在一起。

字节 (byte) 内存中可寻址的最小单元，大多数机器的字节占 8 位。

类成员 (class member) 类的组成部分。

复合类型 (compound type) 是一种类型，它的定义以其他类型为基础。

const 是一种类型修饰符，用于说明永不改变的对象。`const` 对象一旦定义就无法再

赋新值，所以必须初始化。

常量指针 (const pointer) 是一种指针，它的值永不改变。

常量引用 (const reference) 是一种习惯叫法，含义是指向常量的引用。

常量表达式 (const expression) 能在编译时计算并获取结果的表达式。

constexpr 是一种函数，用于代表一条常量表达式。6.5.2 节（第 214 页）将介绍 `constexpr` 函数。

转换 (conversion) 一种类型的值转变成另外一种类型值的过程。C++ 语言支持内置类型之间的转换。

数据成员 (data member) 组成对象的数据元素，类的每个对象都有类的数据成员的一份拷贝。数据成员可以在类内部声明的同时初始化。

声明 (declaration) 声称存在一个变量、函数或是别处定义的类型。名字必须在定义或声明之后才能使用。

声明符 (declarator) 是声明的一部分，包括被定义的名字和类型修饰符，其中类型修饰符可以有也可以没有。

decltype 是一个类型说明符，从变量或表达式推断得到类型。

默认初始化 (default initialization) 当对象未被显式地赋予初始值时执行的初始化行

79